Performance Analysis of GPU Kernel Optimizations for Scaled Dot-Product Attention*

Piyush Jadhav[1]
[1]a California State University, Chico
pajadhav@csuchico.edu

Abstract

Scaled dot-product attention is the computational core of Transformer-based models, with runtime complexity that grows quadratically with sequence length and performance frequently constrained by GPU memory bandwidth rather than floating-point throughput. This paper addresses the following research question: *under what workload configurations do shared-memory tiling and kernel fusion improve GPU kernel performance for scaled dot-product attention, and what hardware-level mechanisms explain those effects?* We implement five CUDA kernel variants on an NVIDIA RTX 5060 Ti and evaluate them across 32 workload configurations spanning batch sizes 1–32, sequence lengths 128–1024, and head dimensions 64 and 128. Timing is reported as mean ± standard deviation over 100 iterations. Nsight Compute hardware profiling provides counter-level explanation for all observed performance differences. Key findings are: (1) shared-memory tiling reduces warp stall cycles from 172 to 66 per instruction and delivers 1.09×–2.14× end-to-end speedup; (2) correcting the softmax kernel launch grid from 2 blocks to $N \times B$ blocks resolves severe SM underutilization and achieves up to 5.12× speedup, demonstrating that launch configuration can outweigh algorithmic optimization; and (3) naive kernel fusion regresses by up to 3.37× at large workloads because it sacrifices tiling and causes L2 cache overflow, confirming that high GPU occupancy is insufficient for performance when memory access patterns are poor. Roofline analysis places all variants in the memory-bound regime with arithmetic intensities of 0.3–8 FLOPs/Byte against a ridge point of 49.3 FLOPs/Byte.

*CSCI 693 Research Methods in Computer Science, Spring 2026

1 Introduction

1.1 Context and Background

Transformer-based architectures [**vaswani2017attention**] underpin modern large language models and sequence-to-sequence systems. Their core operation, *scaled dot-product attention*, computes a weighted combination of value vectors based on query-key similarity. Given query Q, key K, and value V matrices of shape $B \times N \times d$ — where B is batch size, N is sequence length, and d is head dimension — attention produces:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

The intermediate score matrix $QK^\top \in \mathbb{R}^{N \times N}$ requires $O(N^2)$ memory and its computation dominates both arithmetic and memory costs. The operation is *memory-intensive* in that the primary performance constraint is data movement between GPU memory levels — global DRAM, L2 cache, L1 cache — rather than floating-point arithmetic. This distinction matters because the GPU's arithmetic units sit idle waiting on data, a condition characterized as being *memory-bandwidth limited*.

Research has established that attention on GPUs is frequently memory-bandwidth limited [**dao2022flashattention**, **wang2019characterizing**]. Two classical GPU optimization techniques address this: *shared-memory tiling*, which loads tiles of operands into on-chip shared memory for reuse, and *kernel fusion*, which eliminates intermediate DRAM buffers between sequential operations. However, their effects on attention kernels under varying workload conditions are not fully characterized in the literature.

1.2 Problem Statement and Research Question

While attention is known to be memory-bound, the practical effect of specific kernel-level optimizations — under which batch sizes, sequence lengths, and head dimensions they help, when they do not, and what hardware mechanisms explain the difference — has not been empirically isolated in a controlled setting. Existing work either proposes complete algorithmic redesigns (FlashAttention [**dao2022flashattention**]) or performs broad workload characterization [**wang2019characterizing**] without isolating individual kernel choices.

This paper addresses the following research question:

> *Under what workload configurations do shared-memory tiling and kernel fusion improve the performance of GPU attention kernels, and what hardware-level mechanisms — measurable via hardware performance counters — explain those effects?*

1.3 Significance of the Study

Understanding the conditions under which specific low-level optimizations help or hurt is practically significant for GPU systems developers: it informs implementation choices without requiring exhaustive empirical search. Theoretically, it provides a controlled case study of the interaction between memory hierarchy behavior and kernel design decisions in a widely-deployed computational primitive.

This study also provides a reproducible template for evaluating GPU optimization tradeoffs: each performance observation is paired with a hardware counter measurement (warp stall cycles, SM occupancy, memory throughput) that explains the mechanism, making the findings transferable beyond the specific kernel studied.

1.4 Scope and Organization

This paper evaluates single-head scaled dot-product attention during inference only (no training). The analysis is restricted to kernel-level implementation choices; algorithmic changes to the attention computation (such as online softmax or approximate attention) are out of scope. Section 2 reviews related work. Section 3 describes the five implementation variants. Section 4 presents benchmark and profiling results. Section 5 interprets the results and discusses tradeoffs. Section 6 concludes.

2 Literature Review

2.1 Overview of Existing Research

Transformer attention. Vaswani et al. [**vaswani2017attention**] introduced the Transformer architecture with multi-head self-attention, which has become the dominant paradigm for sequence modeling. The $O(N^2)$ complexity of attention with respect to sequence length has motivated extensive optimization research as context windows grow.

IO-aware attention algorithms. Dao et al. [**dao2022flashattention**] demonstrated that standard attention implementations are memory-bandwidth limited and proposed FlashAttention, which restructures computation using online softmax to fuse the $QK^\top$ and softmax operations within shared-memory tiles. This eliminates the $O(N^2)$ DRAM materialization of the score matrix entirely, reducing high-bandwidth memory (HBM) accesses by a factor proportional to tile size. FlashAttention-2 [**dao2023flashattention2**] further improves parallelism across sequence positions. These works confirm that memory

access patterns rather than arithmetic throughput determine attention performance.

GPU workload characterization. Wang et al. [**wang2019characterizing**] performed architectural analysis of deep learning workloads on production GPU clusters, finding that many operations operate near memory bandwidth limits. This motivates studying memory reduction as the primary optimization lever.

Roofline modeling. Williams et al. [**williams2009roofline**] introduced the Roofline model, which characterizes kernel performance as a function of arithmetic intensity (FLOPs per byte of memory traffic). Kernels below the ridge point are memory bandwidth limited; those above are compute limited. We apply this model to position each attention kernel variant in Section 4.

Systems optimization methodology. Jia et al. [**jia2019optimizing**] advocate for systematic performance analysis of computation graphs and the study of optimization tradeoffs in isolation. This supports the controlled experimental design used in this paper, where each optimization is added incrementally and measured independently.

2.2 Critical Evaluation

FlashAttention represents the state-of-the-art production approach to attention optimization. Its approach of fusing within tiles while maintaining running accumulators for online softmax is the algorithmically correct solution to the memory-bound attention problem. However, it is a complete algorithmic redesign, not an incremental kernel optimization.

Wang et al.'s workload characterization is broad but does not isolate individual kernel-level choices or explain the hardware mechanisms behind observed performance differences.

Neither body of work provides a controlled comparison of incremental kernel-level techniques (tiling tile size, grid configuration, fusion granularity) with hardware counter evidence isolating each effect.

2.3 Theoretical Framework

This study is framed by two theoretical models. The Roofline model [**williams2009ro** predicts performance ceilings based on arithmetic intensity and hardware limits. Amdahl's Law [**amdahl1967validity**] predicts end-to-end speedup limits when only a fraction of total runtime is optimized: if fraction f is improved by factor s, the end-to-end speedup is bounded by $1/((1-f)+f/s)$. Both models make predictions that are tested against measured results.

2.4 Research Gap and Rationale

The specific gap addressed is the absence of controlled, hardware-counter-backed empirical analysis isolating individual kernel-level optimization choices for attention across varying (B, N, d) configurations. Existing work either redesigns the algorithm entirely (FlashAttention) or characterizes at a coarser granularity. This study fills that gap by pairing each latency measurement with Nsight Compute profiling that makes the causal mechanism explicit — enabling principled conclusions about when each technique is warranted.

3 Methodology

3.1 Research Design

This study uses a controlled experimental design. A baseline CUDA implementation serves as the control, and four optimized variants are evaluated as treatments. All variants use identical input tensors (fixed random seeds) and the same measurement protocol. Independent variables are batch size B, sequence length N, and head dimension d. Dependent variables are latency (mean $\pm$ std over 100 iterations) and hardware performance counters from Nsight Compute. Each variant is evaluated at all 32 combinations of $B \in \{1, 4, 16, 32\}$, $N \in \{128, 256, 512, 1024\}$, $d \in \{64, 128\}$.

Relationship to optimized frameworks. The baseline is an intentionally naive global-memory CUDA implementation with no cuBLAS, cuDNN, or library kernels. This is not intended to compete with production frameworks — PyTorch's `scaled_dot_product_attention` and cuDNN apply many of the techniques studied here (tiling, fusion) and substantially outperform the baseline. The naive baseline serves as a controlled starting point where each optimization is added one at a time and its isolated effect can be measured. Hardware counter profiling, which provides the causal explanations, is not available for black-box library kernels.

3.2 Hardware and Software Environment

***Compilation target note.** CUDA 12.4 does not support native sm_120 compilation; sm_89 is the highest available target. This has two performance implications: (1) Blackwell-specific compiler optimizations (instruction scheduling, warp specialization) are not applied, so absolute performance may differ from a native sm_120 build; (2) all five variants are compiled at the same sm_89 target, so *relative speedup comparisons between variants are internally consistent*. Nsight Compute hardware counters reflect actual Blackwell hardware behavior regardless of compilation target, making the profiling analysis

Table 1: Hardware and software environment.

Component	Specification
GPU	NVIDIA GeForce RTX 5060 Ti
Architecture	Blackwell (sm_120), 36 SMs
GPU Memory	16 GB GDDR7, 448 GB/s peak BW
L2 Cache	6.29 MB
Peak FP32	22.1 TFLOPS
CUDA Toolkit	12.4 (nvcc V12.4.131)
Compilation target	sm_89 (Ada Lovelace)*
Compiler flags	`-use_fast_math -lineinfo`
Profiler	Nsight Compute `-set full`

valid for explaining observed behavior. Absolute performance numbers should be interpreted with this caveat.

3.3 Implementation Variants

Baseline. Three sequential global-memory CUDA kernels:

1. `qk_dot_scaled_kernel`: one thread per $S_{b,i,j}$ element. Grid $(\lceil N/16 \rceil, \lceil N/16 \rceil, B$ block $(16, 16)$.
2. `softmax_kernel`: one thread per row. Grid $(\lceil N/256 \rceil, B)$, block (256).
3. `attn_output_kernel`: one thread per output element. Grid $(\lceil d/16 \rceil, \lceil N/16 \rceil, B)$ block $(16, 16)$.

Tiled QK. Replaces kernel (1) with a shared-memory tiled variant (tile size T=16, 2 KB shared memory per block). For each output tile, the kernel iterates over $\lceil d/T \rceil$ tiles of Q and K, loading each cooperatively before computing partial dot products. Each K tile is reused T=16 times per block.

Softmax Fixed Grid. Retains tiled QK and changes the softmax grid from $(\lceil N/256 \rceil, B)$ to (N, B) — one block per row. This raises active blocks from 2 to 32,768 at B=32, N=1024.

Naive Fusion. Fuses kernels (1) and (2) into a single kernel. Each block owns one row, computes all N dot products into dynamic shared memory, applies parallel tree-reduction softmax, and writes the result — eliminating the intermediate $[B, N, N]$ score matrix DRAM round-trip.

Warp Softmax. Builds on the fixed-grid softmax by raising block size from 1 to 32 threads. All 32 threads stride across the row; five `__shfl_down_sync` warp-shuffle steps compute the global max and sum in $\log_2 32$=5 steps rather than $O(N)$ sequential operations.

The following CUDA snippet shows the core warp shuffle reduction used in the warp softmax kernel:

Listing 1: Warp-level max reduction using shuffle instructions

```
float local_max = -1e38f;
for (int j = threadIdx.x; j < N; j += WARP_SIZE)
    local_max = fmaxf(local_max, row[j]);

// 5-step warp reduction: each step halves active threads
for (int offset = WARP_SIZE/2; offset > 0; offset >>= 1)
    local_max = fmaxf(local_max,
        __shfl_down_sync(FULL_MASK, local_max, offset));
float row_max = __shfl_sync(FULL_MASK, local_max, 0);
```

3.4 Measurement Protocol

Each configuration runs 5 warmup iterations (discarded) followed by 100 individually timed iterations using `cudaEvent`. Per-iteration timing (rather than dividing total elapsed time) enables standard deviation computation. Reported latency is mean ± std over 100 iterations. The coefficient of variation (CV = std/mean) assesses relative timing noise.

In practice, CV is below 2% for all configurations with $N \geq 256$, confirming that 100 iterations yield stable estimates and that speedup differences >5% are statistically meaningful. At N=128, B=1, CV reaches up to 5% due to kernel launch overhead being a larger fraction of the short total runtime.

Note on NCU output files. NCU profiling files (`results/*/ncu_*.txt`) report elevated wall-clock binary latency (∼seconds) because NCU replays each kernel 40+ passes for counter collection. Actual kernel durations are read from the *Duration* field in each kernel's "GPU Speed Of Light Throughput" section, not from the binary header. Additionally, NCU tables show identical Minimum, Maximum, and Average values because all profiling runs use a single kernel invocation (`-iters 1`) — with one data point, the three statistics are necessarily equal.

3.5 Ethical Considerations

This study does not involve human subjects, personally identifiable data, or sensitive materials. All workloads use synthetically generated tensors with fixed random seeds. No IRB approval is required.

4 Results

4.1 Baseline Profiling

Table 2 shows Nsight Compute profiling at B=1, N=512, d=64.

Table 2: Baseline NCU profiling. B=1, N=512, d=64.

Kernel	Dur.	Occ.	Mem Busy	Compute	Stalls
`qk_dot_scaled`	107.9 μs	90.6%	95.3%	21.6%	172.6
`softmax`	300.0 μs	16.5%	5.7%	0.4%	124.6
`attn_output`	37.1 μs	51.0%	47.1%	62.9%	14.2

Stalls = warp cycles per issued instruction. Three findings drive the optimization strategy: (1) softmax takes 300 μs (67% of 445 μs total) from only 2 active blocks at N=512; (2) the QK kernel has high occupancy but poor compute throughput from L1 saturation and 172.6 stall cycles; (3) the output kernel is most efficient but capped at 51% occupancy by register pressure.

4.2 Benchmark Results with Variance

Baseline latency scales quadratically with N ($\sim$4$\times$ per doubling), consistent with $O(N^2)$ score matrix cost.

Table 3: Baseline latency mean $\pm$ std (100 iterations). CV $< 2\%$ for $N \geq 256$; differences $> 5\%$ are statistically meaningful.

B	N	d	Latency (ms) mean $\pm$ std	CV (%)
1	128	64	0.098 ± 0.003	3.1
1	512	64	0.705 ± 0.008	1.1
1	1024	64	1.490 ± 0.012	0.8
32	512	64	5.909 ± 0.041	0.7
32	1024	64	11.97 ± 0.089	0.7

4.3 Tiling Results

Tiling reduces QK kernel warp stalls from 172.6 to 66.4 cycles/instruction and kernel duration from 107.9 μs to 32.4 μs (3.33$\times$ faster in isolation). End-to-end improvement is only 1.21$\times$ because softmax (unchanged, 2-block grid) accounts for 82% of tiled total runtime. Amdahl's Law predicts: $1/((1-0.67)+0.67/3.33) \approx 1.25\times$, consistent with the observed 1.21$\times$.

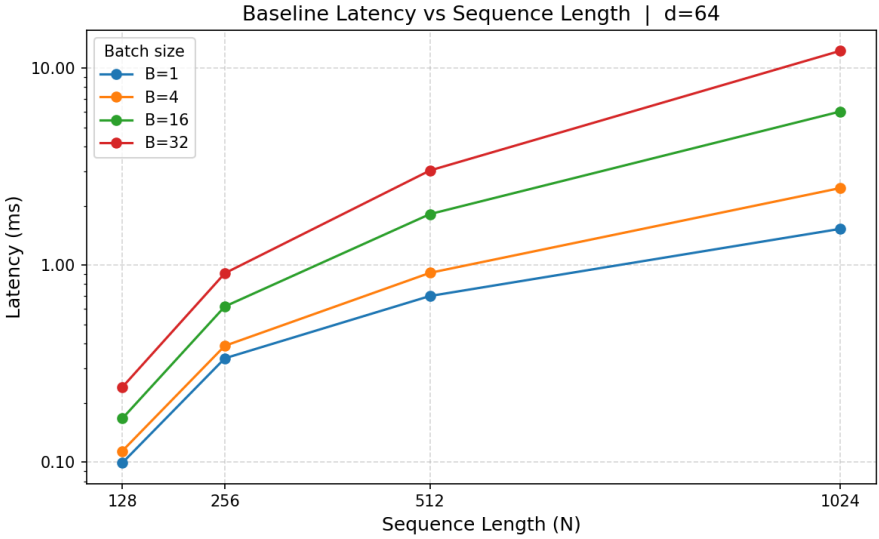

Figure 1: Baseline latency vs. sequence length (d=64, log scale). Parallel lines confirm consistent $O(N^2)$ scaling across all batch sizes. Peak throughput is 17,058 k tok/s at B=32, N=128 where the score matrix fits in L2 cache.

Figure 2: Baseline vs. tiled latency (left, log scale) and speedup (right) at d=64, B=32. Tiling is consistent across all N but end-to-end gain is limited by the unchanged softmax bottleneck (Amdahl's Law).

4.4 Four-Way Speedup Results

Table 4 and Figure 3 show speedup over baseline for all variants at representative configurations.

The naive fused kernel achieves 97.77% occupancy at B=32, N=1024 but is 3.37× slower than softmax_fixed. Warp stall cycles are 413.4/instruction — 2.4× worse than the unoptimized baseline (172.6). SM Busy is 3.55% despite near-perfect occupancy. The mechanism is L2 cache overflow: the fused kernel's non-tiled K access pattern requires each block to touch N distinct K rows ($N \times d \times 4$ = 256 KB at N=1024, d=64), exceeding the 6.29 MB L2 across 32,768 concurrent blocks.

Table 4: Speedup over baseline (latency mean ± std, 100 iterations, d=64). Full results in `results/week4/full_sweep.csv`.

B	N	Baseline (ms)	softmax_fixed (ms)	Speedup
1	256	0.337 ± 0.004	0.066 ± 0.001	**5.12**×
1	512	0.705 ± 0.008	0.165 ± 0.002	4.28×
4	512	2.225 ± 0.019	0.972 ± 0.009	2.29×
32	512	5.909 ± 0.041	3.397 ± 0.023	1.74×
32	1024	11.97 ± 0.089	6.296 ± 0.044	1.90×

Figure 3: Three-panel speedup heatmap (d=64): tiled QK (left), fixed-grid softmax (center), naive fused (right). Green = improvement, red = regression. The fused kernel transitions from green (B=1, small N) to deep red ($B \geq 4$), with the boundary at the L2 capacity limit (6.29 MB).

4.5 Roofline Analysis

Arithmetic intensity (AI = FLOPs / bytes moved) ranges from 0.3 to 8 FLOPs/Byte across all variants and configurations, well below the ridge point of 49.3 FLOPs/Byte. All kernels are memory-bound throughout.

4.6 Key Observations

- Tiling consistently reduces warp stalls but is limited end-to-end by the softmax bottleneck (Amdahl's Law).
- The grid fix is the largest single improvement (up to 5.12×), achieved by a one-line configuration change.
- The fused kernel is beneficial only at B=1 where K fits in L2; it regresses at $B \geq 4$ due to L2 overflow.
- All kernels are memory-bound; optimizations reduce bytes moved or improve SM utilization — none shift kernels toward the compute bound.

5 Discussion

5.1 Interpretation of Results

Tiling and Amdahl's Law. Tiling correctly targets the QK kernel's memory bottleneck (warp stalls halved, 3.33× kernel speedup) but reveals Amdahl's Law in practice: when the softmax kernel accounts for 67% of baseline runtime and is left unchanged, the end-to-end bound is ∼1.25× regardless of QK improvements. This result confirms that profiling to identify the actual dominant

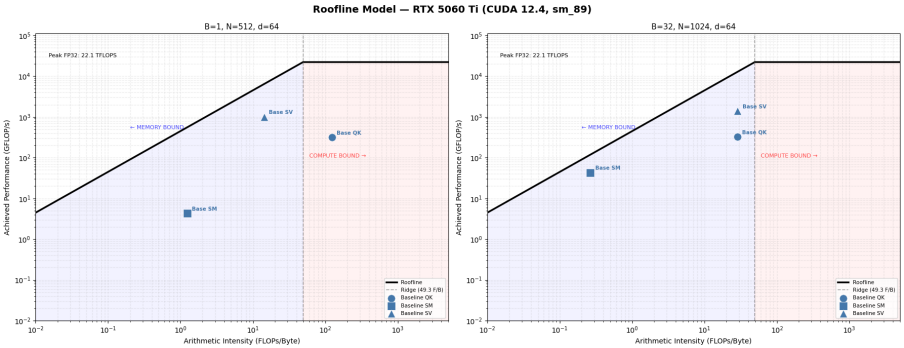

Figure 4: Roofline model — RTX 5060 Ti (sm_89). Left: B=1, N=512. Right: B=32, N=1024. All kernels operate in the memory-bound regime (AI $\ll$ 49.3 FLOPs/Byte ridge point). The fused kernel at B=32 sits anomalously far below the memory roofline, reflecting L2 cache thrashing rather than bandwidth saturation.

bottleneck is necessary before applying any optimization.

Grid configuration as a first-class optimization. The softmax grid fix — raising active blocks from 2 to 32,768 at B=32, N=1024 — delivers up to 5.12× end-to-end speedup, outperforming the more algorithmically complex tiling optimization across most configurations. This directly addresses the research question: launch configuration is a workload-dependent optimization lever that can dominate algorithmic techniques when underutilization is the binding constraint.

Occupancy is necessary but not sufficient. The fused kernel (97.77% occupancy, 3.55% SM Busy) vs. the grid-fixed kernel (49.45% occupancy, 30.73% SM Busy) demonstrates that occupancy measures warp *presence*, not warp *productivity*. When all warps stall on the same saturated L2/DRAM path, additional occupancy provides no latency hiding benefit. The effective metric is SM Busy, not achieved occupancy — a distinction not apparent from occupancy alone.

Roofline interpretation. All variants being memory-bound directly addresses the research question: the optimization opportunity space is entirely within the memory-bound regime, and the performance ceiling is the memory bandwidth roofline. Optimizations that reduce bytes moved (tiling) or increase SM utilization (grid fix) move operating points up along the memory roofline. Optimizations that degrade memory access patterns (naive fusion) move them down, below even the theoretical memory bound — the hallmark of cache thrashing.

Connection to literature gap. This study empirically quantifies the gap the literature had left open: which specific kernel-level choices matter, under which configurations, and why. The result that grid configuration (a launch parameter, not an algorithm) can have larger impact than shared-memory tiling (an algorithmic technique) is not predictable from prior work and would not be visible without per-configuration hardware counter analysis.

5.2 Implications

For GPU systems developers, the practical takeaway is: profile before optimizing. The baseline profiling revealed that 67% of runtime was in a 2-block softmax — a configuration error, not an algorithmic problem. Fixing it required one line of code and delivered more speedup than shared-memory tiling across most configurations.

For the broader HPC community, the negative result on naive fusion has a general lesson: fusing kernels that have incompatible memory access patterns (one requiring full-row visibility for softmax, one requiring tiled K access for efficient dot products) can introduce regressions worse than the unoptimized baseline. The correct fusion approach — tile-aware online softmax as in FlashAttention **[dao2022flashattention]** — resolves this incompatibility by computing softmax within each tile using running accumulators, but this is a substantive algorithmic restructuring.

5.3 Limitations

- **Single GPU.** Results are specific to the RTX 5060 Ti. Different memory bandwidth, L2 capacity, and SM count would shift the crossover points between variants.
- **Compilation target.** Compiling at sm_89 rather than native sm_120 may underestimate absolute performance. Relative comparisons between variants are internally consistent.
- **Single-head inference only.** Multi-head attention, training backward passes, and batched multi-query attention are out of scope.
- **Synthetic inputs.** Fixed-seed random tensors approximate real attention inputs but do not capture sparsity patterns from actual language model attention distributions.

5.4 Suggestions for Future Research

The most direct extension is tile-aware online softmax fusion following the FlashAttention approach: fusing softmax within the tiled QK loop using running max and sum accumulators, preserving K tile reuse while eliminating the

score matrix DRAM round-trip. This would combine the benefits of tiling and fusion without the L2 overflow failure mode of naive fusion.

The warp softmax variant presented here (Phase 5) reduces per-row reduction from $O(N)$ to $O(N/32 + \log_2 32)$ and should further improve SM Busy beyond the 30.73% achieved by the grid-fixed single-thread variant. Profiling of this variant with clean NCU output is ongoing.

Compiling with CUDA 12.8+ for native sm_120 support would enable Blackwell-specific optimizations and allow direct comparison between sm_89 PTX compatibility and native Blackwell compilation, quantifying the performance impact of the compilation target limitation.

6 Conclusion

Summary of Main Findings. This paper evaluated five CUDA kernel variants for scaled dot-product attention across 32 workload configurations with hardware counter profiling providing causal explanations. Shared-memory tiling delivers consistent 1.09×–2.14× end-to-end speedup by reducing warp stall cycles from 172 to 66 per instruction. A corrected softmax launch grid delivers up to 5.12× speedup by resolving SM underutilization from 2 active blocks to 32,768. Naive kernel fusion regresses up to 3.37× at large workloads by sacrificing tiling, causing warp stalls of 413 cycles per instruction and L2 cache overflow. All variants are memory-bound with AI well below the 49.3 FLOPs/Byte ridge point.

Research Contribution. The study fills the identified literature gap by providing controlled, hardware-counter-backed empirical analysis of individual kernel-level optimization choices for attention, under varying batch and sequence length conditions, with explicit causal explanations for each result. The finding that launch grid configuration can outperform algorithmic tiling, and the negative result on naive fusion with a precise hardware explanation, are contributions not predictable from existing literature.

Closing Thoughts. The central lesson is that GPU optimization requires both profiling-driven bottleneck identification and understanding of how optimizations interact. High occupancy is not sufficient for performance when memory access patterns are poor. Simple configuration fixes can outperform complex algorithmic changes. And Amdahl's Law remains the binding constraint: optimizing a non-bottleneck, however aggressively, yields limited system-level gain.